

Prueba Backend

1. Manipulador manifests

En nuestro equipo, estamos desarrollando un servicio clave para nuestra plataforma de streaming. Este servicio se encarga de procesar y manipular manifiestos HLS (**.m3u8**) para optimizar la entrega de video a nuestros clientes.

Un manifest HLS es un archivo **.m3u8** que describe:

- **Master playlist:** lista de variantes (diferentes resoluciones/bitrates) con metadatos como ancho de banda, resolución y codecs.
- **Media playlist:** lista secuencial de segmentos de video/audio.

Recurso: <https://www.cloudflare.com/es-es/learning/video/what-is-http-live-streaming/>

Actualmente, contamos con un prototipo básico funcional que permite filtrar las *playlists* (variantes de stream) de un manifiesto basándose en la **resolución** de vídeo.

Tu tarea es tomar este prototipo y evolucionarlo para que esté listo para un lanzamiento a producción.

Requerimientos:

1. Extender la funcionalidad actual para que, además de filtrar por resolución, se pueda filtrar por un **rango de bitrate** (mínimo y máximo).
2. Tener en cuenta que el equipo de producto tiene planteado introducir 3 nuevos criterios en el futuro.

Recursos:

- Se provee el servicio dentro de la carpeta *Backend* que contiene el código fuente junto a un README con la explicación del mismo.
 - Se provee un git local inicial, se espera que en el entregable se incluya el .git local actualizado. Es un proyecto confidencial del cliente XYZ el cual no se puede publicar en cuentas de github u otro repositorio.
- Ejemplos manifests HLS en vivo:
 - <https://test-streams.mux.dev/x36hzz/x36hzz.m3u8>
 - https://test-streams.mux.dev/x36hzz/url_6/193039199_mp4_h264_aac_hq_7.m3u8

2. Reconstrucción de playlist

Para garantizar una alta disponibilidad, nuestro sistema de streaming obtiene las listas de segmentos de video (chunklists) desde múltiples servidores o CDNs en paralelo.

Ocasionalmente, uno de estos servidores puede fallar o tener latencia, resultando en una "chunklist" incompleta, con "huecos" o segmentos faltantes.

Nuestro reproductor de video necesita una lista completa y ordenada de todos los segmentos para funcionar correctamente.

Ejemplo de chunklist con huecos (faltan los IDs 2 y 3):

```
#EXTINF:8.334,  
media_w163907510_chunklist_1.ts  
#EXTINF:8.334,  
media_w163907510_chunklist_4.ts  
#EXTINF:8.333,  
media_w163907510_chunklist_5.ts  
#EXTINF:8.333,  
media_w163907510_chunklist_6.ts  
#EXTINF:8.334,  
media_w163907510_chunklist_7.ts
```

Requerimientos:

Implementar un módulo que dado un conjunto de archivos con las chunklists, seleccione el subconjunto mínimo de ellas necesario para cubrir la totalidad de los IDs de segmentos.

Ejemplo:

Supongamos que la secuencia de video completa requiere los segmentos del **1 al 10**. Se te proporciona un conjunto de 5 chunklists candidatas:

- `chunklist_A.m3u8` contiene los IDs: [1, 2, 3, 4, 5]
- `chunklist_B.m3u8` contiene los IDs: [6, 7, 8, 9, 10]
- `chunklist_C.m3u8` contiene los IDs: [1, 3, 5, 7, 9]
- `chunklist_D.m3u8` contiene los IDs: [2, 4, 6, 8, 10]
- `chunklist_E.m3u8` contiene los IDs: [5, 6, 7]

Análisis del caso:

- Una solución **válida** podría ser [`chunklist_A`, `chunklist_E`, `chunklist_B`]. Si combinas **A** y **E**, tienes los segmentos del 1 al 7. Al añadir **B**, cubres el resto. Esta solución usa **3 archivos**.
- Sin embargo, una solución **óptima** es [`chunklist_C`, `chunklist_D`], ya que cubre todos los segmentos del 1 al 10 utilizando únicamente **2 archivos**.

Salida esperada para este ejemplo:

JSON

```
JavaScript  
["chunklist_C.m3u8", "chunklist_D.m3u8"]
```

(O también `["chunklist_A.m3u8", "chunklist_B.m3u8"]`, que es otra solución óptima con 2 archivos).

Notas

- El módulo puede implementarse en cualquier lenguaje de programación.
- No es necesario integrarlo a la parte 1.

Recursos:

- Se provee con un conjunto de chunklists para usar como pruebas en la carpeta Manifests.